

Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading

Hanfei Yu
Stevens Institute of Technology
New Jersey, USA
hyu42@stevens.edu

Xingqi Cui*
Rice University
Texas, USA
xc66@rice.edu

Hong Zhang
University of Waterloo
Ontario, Canada
hongzhangblaze@gmail.com

Hao Wang
Rutgers University
New Jersey, USA
hw488@cs.rutgers.edu

Hao Wang
Stevens Institute of Technology
New Jersey, USA
hwang9@stevens.edu

Abstract

Large Language Models (LLMs) have gained immense success in revolutionizing various applications, including content generation, search and recommendation, and AI-assisted operations. To reduce high training costs, Mixture-of-Experts (MoE) architecture has become a popular backbone for modern LLMs. However, despite the benefits, serving MoE-based LLMs experience severe memory inefficiency due to sparsely activated experts. Recent studies propose to offload inactive experts from GPU memory to CPU memory to improve the serving efficiency of MoE models. However, they either incur high inference latency or high model memory footprints due to coarse-grained designs.

To tame the latency-memory trade-off in MoE serving, we present *FineMoE*, a fine-grained expert offloading system for MoE serving that achieves low inference latency with memory efficiency. We design *FineMoE* to extract fine-grained expert selection patterns from MoE models and semantic hints from input prompts to efficiently guide expert prefetching, caching, and offloading decisions. *FineMoE* is prototyped on top of HuggingFace Transformers and deployed on a six-GPU testbed. Experiments with open-source MoE models and real-world workloads show that *FineMoE* reduces inference latency by 47% and improves expert hit rate by 39% over state-of-the-art solutions.

CCS Concepts: • Computing methodologies → Distributed algorithms; Artificial intelligence; Machine learning.

*This work was conducted while Xingqi Cui was a remote intern student, advised by Dr. Hao Wang at the IntelliSys Lab, Stevens Institute of Technology.



This work is licensed under a Creative Commons Attribution 4.0 International License.

EUROSYS '26, April 27–30, 2026, Edinburgh, Scotland UK

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/26/04

<https://doi.org/10.1145/3767295.3769319>

Keywords: Artificial Intelligence, Large Language Model, Mixture-of-Experts, Model Serving, Offloading

ACM Reference Format:

Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. 2026. Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading. In *21st European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland UK. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3767295.3769319>

1 Introduction

Large Language Models (LLMs) have achieved remarkable success in advancing Natural Language Processing (NLP) research and transforming various applications, including content generation [2, 7, 12, 45], search and recommendation [34, 63], and AI-assisted operations [24, 33, 39]. Given the high training costs, modern LLMs have returned to Mixture-of-Experts (MoE) architectures [1, 11, 23, 50, 57, 60] as their backbone implementations. Inside MoE models, each MoE layer comprises a gating network and a collection of experts, with only a subset of experts being activated during computation. This sparse activation mechanism significantly reduces the number of floating point operations (FLOPs), enabling MoE-based LLMs to achieve substantially lower training costs compared to dense LLMs [11, 23, 60].

Despite the computational efficiency, MoE models exhibit substantial memory inefficiency during the serving phase. Though certain model parameters remain inactive during inference, they must still reside in GPU memory to allow for potential future activation. Expert offloading [4, 16, 51, 58] has emerged as a promising strategy to address this issue, which predicts inactive experts and transfers them to CPU memory while retaining only the necessary experts in GPU memory, reducing the overall model memory footprint.

However, existing expert offloading solutions struggle to effectively balance the *latency-memory trade-off* in MoE serving. These approaches either suffer from high inference latency [4, 51] or incur substantial model memory footprints [16, 58]. The key reason is that existing works track expert patterns

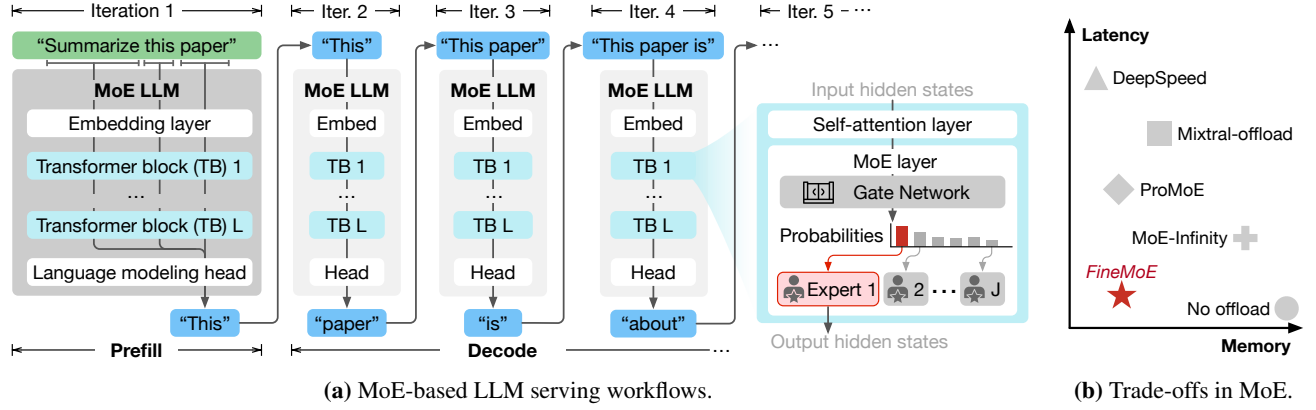


Figure 1. Mixture-of-Experts (MoE) Large Language Model (LLM) serving.

and manage experts in *coarse granularity*. They fail to accurately identify and retain only the necessary experts in GPU memory during inference, resulting in frequent and costly on-demand expert loading [51] and performance degradation.

In this paper, we propose *FineMoE*, a *fine-grained* expert offloading system that tames the latency-memory trade-off in MoE serving. To track and analyze MoE models' expert selection behaviors in fine granularity, we propose a new data structure called *expert map*, which records the iteration-level probability distributions output by the gate network. *FineMoE* uses historical expert maps for comparing expert trajectory similarity to guide offloading.¹ Apart from the expert map, *FineMoE* is designed to track fine-grained input semantic embeddings from individual request prompts processed by the MoE model. Given the collected semantic-based and trajectory-based information, *FineMoE* carefully searches the most accurate expert map for guiding expert prefetching, caching, and offloading through inference iterations. In summary, we make the following contributions:

- We design *FineMoE*, a *fine-grained* expert offloading system that achieves low inference latency while reducing model memory footprints.
- We propose a new data structure, expert map, that tracks fine-grained expert selection behaviors of MoE models. *FineMoE* leverages input semantic embeddings to augment the expert map search to guide expert offloading.
- We prototype *FineMoE* on top of HuggingFace Transformers [55] and deploy it on a six-GPU testbed. Extensive experiments with open-source MoE models and real-world workloads show that *FineMoE* reduces inference latency by 47% and improves expert hit rate by 39% compared to state-of-the-art solutions.

¹In this paper, "trajectory" is defined as the collection of probability distributions over experts observed through layers.

2 Background and Motivation

2.1 LLM Serving

Unlike traditional Deep Learning (DL) model inference, Large Language Model (LLM) serving consists of two consecutive stages: *prefill* and *decode*. Figure 1a illustrates the two stages when an LLM performs inference for a request prompt. In the prefill stage, the LLM first computes the intermediate key-value (KV) states of the prompt tokens, prefills the KV cache [3, 28, 31, 37, 65], and then generates the first answer token. In the decode stage, the LLM sequentially generates the answer to the prompt token-by-token in an auto-regressive manner, where tokens generated previously are used for generating the next token.

The two stages have their own unique characteristics. The prefill stage only requires one *iteration*², processing all tokens in parallel and generating the first answer token. The decode stage spans several iterations, generating one token per iteration until the answer is completed. Due to the different characteristics of the two stages, recent studies [43, 65] have identified that the prefill stage is compute-bounded, while the decode stage is considered memory-bounded. Therefore, people typically quantify the serving performance of LLM two stages using different metrics. For the prefill stage, Time-To-First-Token (TTFT) is commonly employed, which measures the latency from receiving the user request until generating the first answer token. For the decode stage, Tokens-Per-Second (TPS) or Time-Per-Output-Token (TPOT) is used to measure the generation rate of LLM serving.

2.2 MoE-based LLM Serving

By integrating MoE layers in Transformer blocks [54], MoE architectures [61] have emerged as a popular backbone for modern LLMs, such as Mixtral [23], Snowflake Arctic [50], and DeepSeek-MoE [11]. Figure 1a illustrates MoE-based LLMs' typical structures, where feed-forward network (FFN)

²An iteration refers to a single step in auto-regressive inference that generates one new token. The iteration time denotes the end-to-end latency of this step.

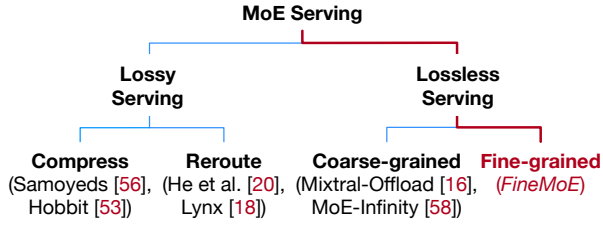


Figure 2. The design space of MoE-based LLM serving.

modules are replaced by MoE layers.³ Each MoE layer consists of a gate network and a set of expert networks. Inside each Transformer block, the self-attention module first calculates the attentions [54] based on input hidden states, and then the gate network determines which expert(s) to activate for computing the output representations. Compared to traditional dense LLMs, MoE-based LLMs only activate a subset of parameters during training and inference, reducing computational overhead while delivering superior generation performance compared to dense LLMs with a comparable number of parameters [1, 11, 23, 50, 57, 60].

Despite the benefits of saving training computations, MoE-based LLM serving still suffers from GPU memory inefficiency as MoE inference requires loading all model parameters into GPU memory, including those inactive experts. Table 1 characterizes three popular MoE models: Mixtral-8×7B [23], Qwen1.5-MoE [60], and Phi-3.5-MoE [1]. During inference, they exhibit 72%, 81%, and 84% inactive parameters, respectively, due to the sparsity of expert activation in MoE. This corresponds to 67, 23, and 70 GB of inactive GPU memory, resulting in low memory efficiency and serving throughput. Therefore, to efficiently serve large MoE models, we must seek a solution to the memory inefficiency inherited from MoE architecture.

2.3 Latency-Memory Trade-Off

Recently, a few studies have been proposed to improve MoE-based LLM serving efficiency. Figure 2 describes the design space in MoE serving. Existing major studies can be categorized into two types: **Lossy serving** applies compression [44], pruning [30], and quantization [27] techniques to the original MoE models to reduce the serving memory requirements. However, this line of work achieves serving efficiency by sacrificing the generation quality. **Lossless serving** focuses on *offloading* model weights (parameters [4, 41] or experts [16, 51, 58]) that are sparsely utilized in temporal or spatial patterns from GPU memory to CPU memory, aiming to preserve reasonable inference latency. Specifically, expert offloading seeks to predict the activation of experts in advance, prefetching or caching only the necessary experts

³For simplicity, we only show the process of one single request prompt.

Table 1. Characteristics of three MoE models.

MoE Models	Parameters (active / total)	Experts Per Layer (active / total)	Num. of Layers
Mixtral-8×7B [23]	12.9B / 46.7B	2 / 8	32
Qwen1.5-MoE [60]	2.7B / 14.3B	4 / 60	24
Phi-3.5-MoE [1]	6.6B / 42B	2 / 16	32

in GPU memory during inference. We opt for lossless serving to design *FineMoE* because this line of methods avoids modifying models, hence assuring generation quality.

However, existing offloading solutions cannot achieve an optimal spot in the latency-memory trade-off when serving MoE-based LLMs. Figure 1b compares the performance (*i.e.*, inference latency and memory footprint) of existing state-of-the-art (SOTA) offloading solutions, which either provide low inference latency but suffer from large memory footprint (*e.g.*, No-offload and MoE-Infinity [58]), or vice versa (*e.g.*, ProMoE [51], Mixtral-Offloading [16], and DeepSpeed-Inference [4]).

The key reason behind this dilemma is that MoE-based decoder-only LLMs have balanced expert routing [51], leaving existing solutions hard to find effective patterns for guiding expert offloading. Existing research has identified two main reasons for this dilemma: First, most MoE-based LLMs are decoder-only architectures, which exhibit uniform expert activation patterns and low expert access skewness compared to encoder-decoder MoE LLMs [18, 51]. Second, recent MoE-based LLMs employ a load-balancing loss [1, 11, 23, 50, 57], which encourages the gate network to distribute tokens more uniformly across experts within each MoE layer, making expert usage more balanced during training. This balanced routing diminishes the predictability of expert patterns, thus making existing solutions ineffective.

2.4 Existing MoE Offloading Solutions

Existing expert offloading approaches [16, 58] rely on **coarse-grained** expert patterns, which are inefficient for guiding offloading. We define coarse-grained information as the expert patterns collected at the request level, where information is aggregated over multiple iterations of a request prompt. For example, MoE-Infinity [58] tracks request-level expert activations. Fine-grained information is defined as the expert patterns observed separately during each inference iteration. Figure 3a shows examples of coarse-grained and fine-grained expert activation heatmaps for Mixtral-8×7B [23]. The heatmap records the expert activations across 32 MoE layers, where each layer contains eight experts and activates two experts out of eight to compute representations. While fine-grained (iteration-level) heatmaps show clear expert activation patterns, the aggregated coarse-grained (request-level) heatmap diminishes predictability.

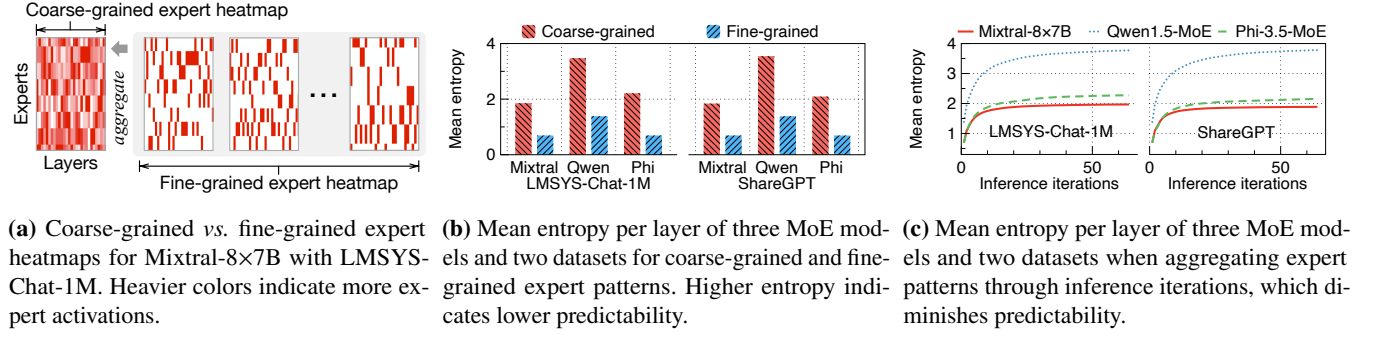


Figure 3. Expert pattern and predictability analysis in coarse granularity (request-level) and fine granularity (iteration-level).

To demonstrate this point, we analyze the Shannon entropy [48] of expert activations per MoE layer for three popular MoE models. Entropy is an essential metric to quantify the uncertainty and unpredictability of variables in information theory. A balanced expert activation pattern (*e.g.*, probability distribution [0.25, 0.25, 0.25, 0.25] of four experts) results in a high entropy, which indicates the pattern is less predictable and harder to select experts. Figure 3b presents the mean entropy computed per layer for three MoE models (Mixtral-8x7B [23], Qwen1.5-MoE [60], and Phi-3.5-MoE [1]) across two realistic datasets LMSYS-Chat-1M [64] and ShareGPT [49]. Coarse-grained expert patterns have significantly higher entropy than fine-grained patterns, meaning that expert patterns in coarse granularity can be less effective for predictions. Figure 3c shows the mean entropy per layer when aggregating expert patterns across inference iterations, where expert selection becomes increasingly unpredictable as generation progresses. Qwen1.5-MoE reaches a higher entropy plateau due to its larger expert selection space (60 experts $\times$ 24 layers). Similarly, Phi-3.5-MoE (16 $\times$ 32) exhibits higher entropy than Mixtral-8x7B (8 $\times$ 32). After about ten iterations, expert patterns become blurred and the entropy plateaus, indicating that further iterations contribute only marginal additional unpredictability. While entropy is low at the beginning of inference, it gradually increases with iterations as more expert activation information is aggregated, thereby becoming more unpredictable.

In contrast to coarse-grained expert offloading solutions, we argue that expert offloading should be carefully guided by **fine-grained** designs: analyzing iteration-level patterns, understanding models' expert selection preferences, and leveraging semantic characteristics of request prompts.

2.5 Problems of Coarse-Grained Offloading

Existing coarse-grained expert offloading solutions exhibit three problems:

1) Insufficient latency-memory trade-off. Existing solutions prefetch and offload experts in coarse granularity, either heavily focusing on reducing inference latency but incurring

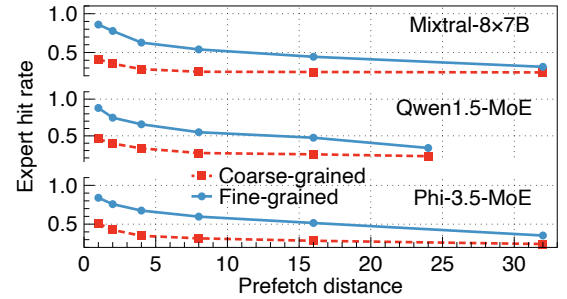


Figure 4. Expert hit rates of coarse-grained and fine-grained expert offloading designs when serving Mixtral-8x7B, Qwen1.5-MoE, and Phi-3.5-MoE with LMSYS-Chat-1M at different prefetch distances, respectively.

large memory footprint [58] or reducing memory footprint but severely increasing inference latency [4, 16].

2) Low expert hit rates. Existing solutions employ coarse-grained expert pattern tracking methods (*e.g.*, Expert Activation Matrix in MoE-Infinity [58]), which produce ineffective expert patterns for guiding offloading decisions, leading to low expert hit rates and high inference latency.

3) Ignorance of MoE models' and prompts' heterogeneity. Existing solutions largely ignore the unique characteristics of different MoE models and input prompts and serve them in a one-fits-all manner [4, 16, 51, 58], which omits opportunities for fine-grained optimizations adaptive to heterogeneous models and prompts in MoE serving.

Figure 4 shows the expert hit rates of serving three popular MoE-based LLMs, Mixtral-8x7B [23], Qwen1.5-MoE [60], and Phi-3.5-MoE [1] using LMSYS-Chat-1M dataset [64] with coarse-grained and fine-grained expert offloading designs at different prefetch distances, respectively. Prefetch distance refers to the number of layers ahead that a prefetch instruction is issued before the target layer activates its experts. By leveraging fine-grained expert offloading, we can achieve

significantly higher expert hit rates over coarse-grained methods and preserve better performance by adapting to varying prefetch distances.

3 FineMoE’s Overview

3.1 Objectives and Challenges

FineMoE is designed to achieve the following three goals:

Memory-efficient MoE serving with minimal inference latency. We have demonstrated that existing expert offloading solutions [16, 51, 58] fail to tame the latency-memory trade-off in MoE serving (§2.3). We aim to achieve both low memory footprint and inference latency by proposing fine-grained expert offloading.

Minimize expert miss due to mispredictions in expert prefetching. Expert prefetching, involving future expert activation predictions, is an essential step in expert offloading solutions. However, a recent study [51] has shown that *expert miss* due to mispredictions can cause high on-demand expert loading delay in inference. We should minimize expert miss and mitigate mispredictions in expert offloading.

Adapt to heterogeneous MoE models and prompts. MoE inference can serve heterogeneous models [11, 23, 50, 57, 60] with varying prompts [49, 64] in real-world scenarios. While existing solutions handle different models and prompts with a one-fits-all design, we should design our expert offloading to adapt to the heterogeneity in MoE serving.

We must address three critical challenges to realize the above objectives:

How to maximize expert hit rate when prefetching and offloading experts? Expert hit rate directly relates to the inference latency. With more experts being hit, fewer experts need to be loaded on demand. We propose a fine-grained expert offloading solution to achieve a high expert hit rate.

How to adapt to different MoE models and prompts? Heterogeneous MoE models and input prompts exhibit unique system and semantic characteristics. We should craft our solution with fine-grained optimizations to enable adaptivity.

How to avoid additional system overheads when managing experts? Our design must not introduce additional system overheads when serving existing MoE LLMs. We apply a series of system optimizations in *FineMoE* to ensure serving efficiency and minimize additional overheads.

3.2 Architecture and Workflow

Figure 5 describes the architecture and workflow of *FineMoE*, which consists of three main components:

Expert Map Store. We record *expert maps*, a new data structure defined in *FineMoE*, to track *fine-grained* expert activation patterns from historical request prompts. expert maps provide nuance expert selection preferences over existing coarse-grained expert tracking methods (e.g., Expert Activation Matrix in MoE-Infinity [58]). The Expert Map

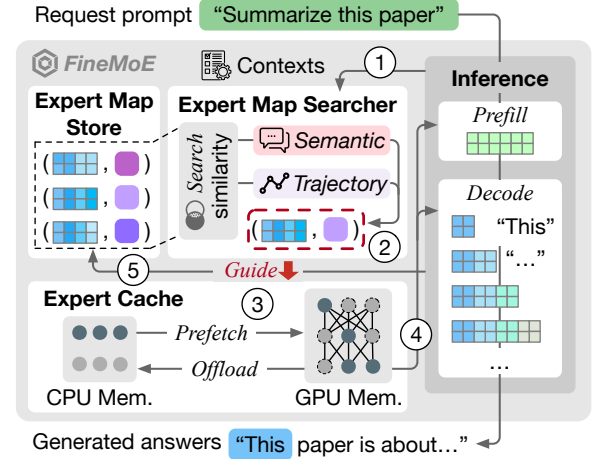


Figure 5. *FineMoE*’s architecture and workflow.

Store dynamically keeps the most useful and unique expert maps for real-time inferences.

Expert Map Searcher. When a request prompt arrives, *FineMoE* searches the Expert Map Store for appropriate expert maps to guide expert prefetching before inference. expert map search is guided by calculating similarity scores in two folds: *semantic* and *trajectory* similarity.

Expert Cache. After receiving the searched expert maps, *FineMoE* prefetches experts from CPU memory to GPU to perform computations in inference. *FineMoE* evicts and offloads low-priority expert weights to CPU memory if exceeding Expert Cache capacity.

FineMoE follows the five steps below to enable memory-efficient MoE serving with minimal inference latency:

Step ①: Inference context collection. Before every inference iteration, *FineMoE* collects necessary *contexts*, such as semantic embeddings and previous expert activation trajectories (§4.1), and feeds them to the Expert Map Searcher for hybrid similarity searching.

Step ②: Expert map similarity searching. After receiving iteration-level contexts, the Expert Map Searcher identifies the most similar expert maps by comparing the input context data with historical context data in the Expert Map Store (§4.2). The retrieved expert maps are forwarded to the Expert Cache to guide expert prefetching and offloading decisions.

Step ③: Guided expert prefetching and offloading. We dynamically compute expert selection thresholds to determine which expert(s) to prefetch and offload in the MoE model guided by the searched expert maps (§4.3). Then, *FineMoE* prefetches the expert weights from CPU to GPU memory and offloads cached experts from GPU to CPU when reaching the cache limit (§4.5).

Step ④: Expert serving. The whole inference process consists of one iteration in the Prefill stage and multiple iterations in the Decode stage. For each MoE layer in every iteration,

FineMoE directly serves the expert required by the gating network if the corresponding weights are available in the GPU memory (defined as an expert hit). Otherwise, *FineMoE* on-demand loads the expert weights from CPU to GPU to perform lossless serving (defined as an expert miss).

Step ⑤: Expert map update. *FineMoE* observes new expert maps produced after each iteration and updates them in the Expert Map Store (§4.4). When reaching the store capacity (e.g., 1K expert maps), *FineMoE* deduplicates the Expert Map Store by identifying and dropping redundant expert maps to maintain diversity, maximizing the possibility of providing effective expert maps for any request prompts.

3.3 Problem Formulation

We consider serving an MoE-based LLM with L MoE layers on a GPU cluster, where each MoE layer has one gating network and J experts. The gating network of each layer selects top $K \in [1, J]$ experts for computation. The MoE model processes and generates answers for a workload consisting of W unique request prompts. Let $[W] := \{1, \dots, w, \dots, W\}$ denote the set of all requests, $[L] := \{1, \dots, l, \dots, L\}$ denote the set of all layers in a MoE model, and $[J] := \{1, \dots, j, \dots, J\}$ denote the set of all experts in a layer, respectively. Each request prompt $w \in [W]$ consists of multiple iterations processed during the prefill and decode stages. Let $E_{l,j}^{(i)}$ denote the j -th expert at the l -th layer in the i -th iteration, where $l \in [L]$, $j \in [J]$, and $i \in [w]$. During each iteration i , we can make at most $L \cdot J$ prefetching decisions. Let $E_{l,j}^{i, \text{cache}}$ and $E_{l,j}^{i, \text{activate}}$ denote the set of cached experts and the set of activated experts for Iteration i , respectively. Hence, we represent the result of whether an expert $E_{l,j}^{(i)} \in E_{l,j}^{i, \text{activate}}$ is missed by $E_{l,j}^{i, \text{cache}}$:

$$R_{l,j}^{(i)} = \begin{cases} 1, & \text{if } (E_{l,j}^{(i)} \in E_{l,j}^{i, \text{activate}}) \wedge (E_{l,j}^{(i)} \notin E_{l,j}^{i, \text{cache}}), \\ 0, & \text{otherwise,} \end{cases}$$

where $R_{l,j}^{(i)} = 1$ means $E_{l,j}^{(i)}$ is a miss and requires on-demand loading from CPU memory. Since all experts in an MoE model are typically designed to have the same weight size, we assume experts' loading time T_e and memory footprint M_e are homogenous.⁴ Therefore, the total on-demand loading latency T is summed across all iterations for each expert during the inference process:

$$T := T_e \cdot \sum_{w \in [W]} \sum_{i \in [w]} \sum_{l \in [L]} \sum_{j \in [J]} R_{l,j}^{(i)}.$$

Finally, employing the above definitions, we formulate the MoE expert offloading as an integer linear programming (ILP)

⁴We only consider selective experts. Some MoE models, such as Qwen1.5-MoE-A2.7B, have a few always-on experts that are not offloadable.

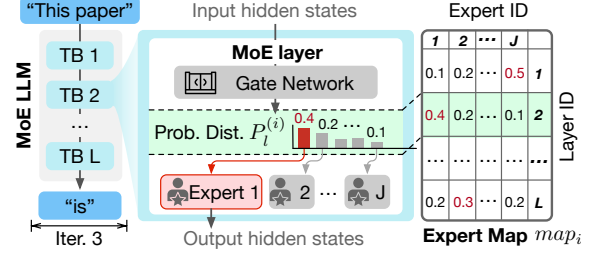


Figure 6. Expert selections tracked by an expert map.

optimization problem:

$$\min_{\{E_{l,j}^{(i)}\}} \left(T_e \cdot \sum_{w \in [W]} \sum_{i \in [w]} \sum_{l \in [L]} \sum_{j \in [J]} R_{l,j}^{(i)} \right) \quad \text{s.t. } |E_{l,j}^{i, \text{cache}}| \leq L \cdot J, \quad \forall i \in [w], \forall w \in [W], \quad (1)$$

$$|E_{l,j}^{i, \text{activate}}| = L \cdot K, \quad \forall i \in [w], \forall w \in [W], \quad (2)$$

$$|E_{l,j}^{i, \text{cache}}| \cdot M_e \leq M, \quad \forall i \in [w], \forall w \in [W]. \quad (3)$$

The objective is to minimize the on-demand loading latency (ideally $T = 0$ with perfect predictions) while limiting the total memory footprint of cached experts to satisfy the available GPU memory M . Constraint 1 denotes the total number of prefetched experts should not exceed the total number of all experts in the MoE model. Constraint 2 represents the total number of activated experts, which must be the same as the total number of top K experts summed across all L layers. Constraint 3 describes the total memory footprint of prefetched experts must be limited by the available GPU memory size. Note that solving the ILP problem is already NP-hard [10], while in reality, prefetching experts always have mispredictions that further complicate the problem. Therefore, we opt for a heuristic-based design for *FineMoE*.

4 FineMoE's Design

4.1 Expert Maps

We propose a new data structure, *Expert Map*, to track expert activation patterns with a fine granularity. Figure 6 depicts the structure of an expert map. During the i -th iteration, the l -th self-attention layer first calculates the attention states. The gate network receives attentions and computes a probability distribution $\mathbf{P}_l^{(i)} \in \mathbb{R}^J$ over all the experts at Layer l :

$$\mathbf{P}_l^{(i)} := \{p_{l,1}^{(i)}, \dots, p_{l,j}^{(i)}, \dots, p_{l,J}^{(i)}\}, \quad \sum_{j \in [J]} p_{l,j}^{(i)} = 1, \quad \forall p_{l,j}^{(i)} \geq 0.$$

Then, top $K \in [1, J]$ experts are selected from $\mathbf{P}_l^{(i)}$ to compute representations for Layer l . We collect the probability distributions $\mathbf{P}_l^{(i)}$ across all L layers to form the expert map of Iteration i :

$$\text{map}_i := \{\mathbf{P}_1^{(i)}, \dots, \mathbf{P}_L^{(i)}\}, \quad l \in [L].$$

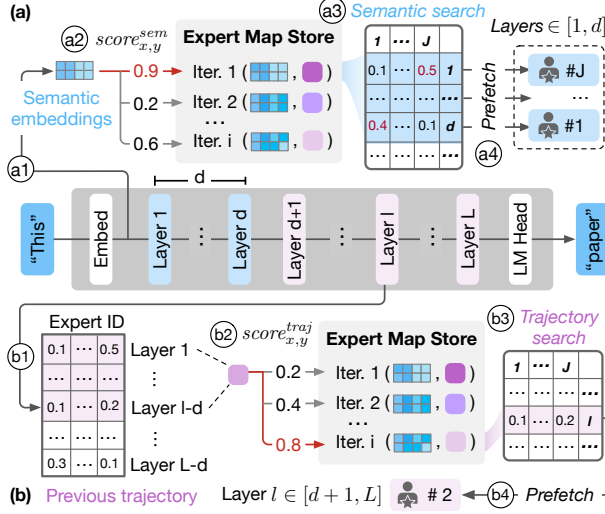


Figure 7. Workflow of *FineMoE*'s expert map search.

By tracking expert maps, we guide *FineMoE* to discover fine-grained expert patterns—the iteration-level expert selection preferences via probability distributions. Intuitively, analyzing probability distributions enables *FineMoE* to not only identify which experts are binarily selected or omitted, but also to assess the confidence or preference assigned to each expert from the perspective of the gate networks.

The design of expert maps has two key advantages over existing coarse-grained expert tracking methods (e.g., MoE-Infinity [58] tracks the request-level expert hit counts). *First*, existing works only focus on *aggregated* request-level expert activations, whereas an expert map tracks individual iterations with detailed expert selections. *Second*, existing works only record the expert hit counts, whereas we track detailed probability distributions. Note that expert maps can easily recover coarse-grained information by applying a top K selection operator to the probability distributions and aggregating expert counts over iterations, therefore generalizing to existing tracking methods.

4.2 Expert Map Search

Given the historical expert maps defined in §4.1, *FineMoE* searches expert maps that provide the most accurate expert activation predictions with two fine-grained metrics: semantic similarity (§4.2.1) and trajectory similarity (§4.2.2). We also show that they are both effective in searching accurate historical expert maps for prediction and offloading (§4.2.3).

Existing solutions [16, 51, 58] *cannot* observe previous expert patterns for prediction and prefetching before the target layer is ready to activate experts for the initial layers $l \in [1, d]$, where l represents the current layer index in an iteration and d is referred as the *prefetch distance*. When predicting and prefetching experts for MoE models, *prefetch distance* is used

to avoid impacting inference latency [51, 62]. Prefetch distance is the number of layers ahead that a prefetch instruction is issued before the target layer activates its experts, similar to the same term in memory prefetching [29]. An ideal prefetch distance should perfectly overlap the prediction and prefetching operation overheads with the inference process.

Therefore, existing approaches [16, 51, 58] typically employ coarse-grained rules to prefetch experts for initial layers $l \in [1, d]$. For example, MoE-Infinity [58] prefetches the most popular experts across all historical data points. Even for layers $l \in [d+1, L]$, existing approaches use coarse-grained (request-level) metrics for predicting and prefetching experts, leading to low offloading accuracy.

In contrast, *FineMoE* leverages fine-grained iteration-level metrics tailored to the prefetch distance d , employing semantic embeddings for layers prior to the prefetch distance and expert trajectories for layers subsequent to it. Figure 7 shows that *FineMoE* employs two fine-grained search approaches to jointly search expert maps for guiding expert prefetching: *Semantic-based expert map search* compares the input embeddings with historical embeddings to find expert maps with similar inputs, whereas *trajectory-based search* observes previous expert trajectories (i.e., probability distributions) and searches for similar expert maps. We combine both semantic and trajectory features to improve *FineMoE*'s map-searching and expert offloading accuracy.

4.2.1 Semantic-based Expert Map Search. Recent studies [25] demonstrate that semantic embeddings, i.e., embedding layer's output after processing raw tokens, can potentially indicate expert selection behaviors. When serving request prompts and recording their expert maps, we record the *semantic embeddings* for each inference iteration. Existing MoE-based LLMs all contain an embedding layer for token semantic encoding, where words or subwords that appear in similar contexts will have similar embeddings [38]. It's natural to extract the semantic embeddings using the output from the model's original embedding layer. Figure 7(a) shows the semantic-based expert map search in four steps: a1) extract semantic embeddings from the embedding layer, a2) compute similarity scores using semantic embeddings with historical data points in the Expert Map Store, a3) search similar expert maps based on similarity scores, and a4) prefetch experts with high probabilities for layers $l \in [1, d]$.

For any input prompts, we compute pairwise cosine similarity $score_{x,y}^{sem} \in \mathbb{R}^{B \times C}$ between the semantic embedding $sem_x^{new} \in \mathbb{R}^{B \times h}$ and the collection of historical semantic embeddings $sem_y^{old} \in \mathbb{R}^{C \times h}$ in the Expert Map Store:

$$score_{x,y}^{sem} := \frac{sem_x^{new} \cdot sem_y^{old}}{\|sem_x^{new}\| \cdot \|sem_y^{old}\|}, \quad x \in [B], y \in [C], \quad (4)$$

where B is the batch size of input prompts, C is the Expert Map Store capacity, and h is the hidden dimension size. Then, for prompt x , the historical Iteration y with the highest score

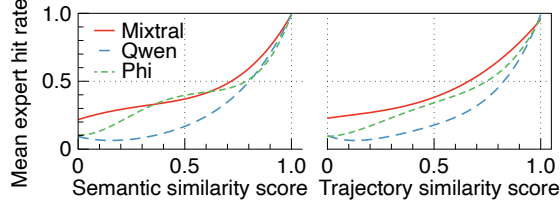


Figure 8. Mean expert hit rates of different semantic and trajectory similarity scores with LMSYS-Chat-1M.

is selected. We use partial expert maps from the selected iteration, $\{P_1^{(y)}, \dots, P_d^{(y)}\} \in \text{map}_y^{\text{old}}$, to guide layers $l \in [1, d]$.

4.2.2 Trajectory-based Expert Map Search. We leverage expert probability trajectories of previous $(l - d)$ layers to search expert maps for layers $l \in [d + 1, L]$. Specifically, when $l = d + 1$, we use the past expert trajectories from Layer 1 for prediction; when $l = d + 2$, we use the past trajectories from Layers 1 and 2; and so on. When $l = L$ (last layer), we use the past trajectories from Layers 1 to $L - d$ for prediction. Figure 7(b) shows the trajectory-based expert search for a layer $l \in [d + 1, L]$ in four steps: b1) collect previous trajectory $\{P_1, \dots, P_{l-d}\}$ from Layers 1 to $l - d$, b2) compute similarity scores using collected trajectories with historical data points in the Expert Map Store, b3) search similar expert maps based on similarity scores, and b4) prefetch experts with high probabilities for the layer $l \in [d + 1, L]$. We repeat this process until the last layer (Layer L) is completed.

Similar to the semantic-based search, we compute pairwise cosine similarity $\text{score}^{\text{traj}} \in \mathbb{R}^{B \times C}$ between the observed trajectories, $\text{map}^{\text{new}} \in \mathbb{R}^{B \times (l-d)J}$, and the collection of historical expert maps, $\text{map}^{\text{old}} \in \mathbb{R}^{C \times (l-d)J}$, in the Expert Map Store:

$$\text{score}_{x,y}^{\text{traj}} := \frac{\text{map}_x^{\text{new}} \cdot \text{map}_y^{\text{old}}}{\|\text{map}_x^{\text{new}}\| \cdot \|\text{map}_y^{\text{old}}\|}, \quad x \in [B], y \in [C]. \quad (5)$$

We select the historical iteration with the highest score. Then, we use $P_l^{(y)} \in \text{map}_y^{\text{old}}$ from the selected expert map to guide the expert prefetching for the target layer $l \in [d + 1, L]$.

By combining the two expert map search methods, we carefully customize the map that guides expert prefetching for every inference iteration in MoE serving. With this design, expert map search introduces negligible overhead to the end-to-end inference latency, which we demonstrate in §6.8.

4.2.3 Effectiveness of Semantic and Trajectory Similarity. To verify how semantic and trajectory similarity scores can guide expert offloading, we run three MoE models (Mixtral-8×7B, Qwen1.5-MoE, and Phi-3.5-MoE) with two datasets (LMSYS-Chat-1M and ShareGPT). For each model and dataset, we first run prompts and record their semantic embeddings and expert trajectories, where each prompt generates one data point consisting of a semantic embedding and an expert map. Then, we exhaust all pairwise cases by calculating their

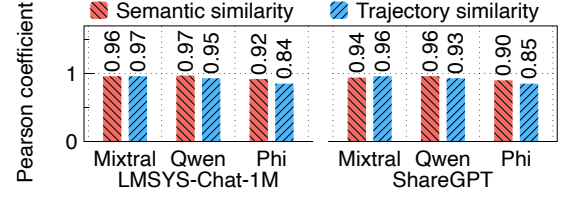


Figure 9. Pearson correlation coefficients between semantic and trajectory similarity scores and expert hit rates.

semantic and trajectory similarity and expert hit rate (*i.e.*, overlapped expert ratio). Figure 8 shows the mean expert hit rates of different semantic and trajectory similarity scores for three MoE models with LMSYS-Chat-1M. Both semantic and trajectory similarity can effectively indicate the accuracy of historical prompts or expert maps for offloading.

To statistically quantify the correlations between similarity score and expert hit rate, we calculate the Pearson correlation coefficients [9] using all paired semantic and trajectory similarity scores and corresponding expert hit rates in Figure 8. The Pearson coefficient is commonly used to measure correlations between variables, where a coefficient close to 1 indicates a strong positive correlation and a coefficient close to 0 means a weak correlation. Figure 9 shows the Pearson coefficients between similarity score and expert hit rate with three MoE models and two datasets. The results show that high similarity scores potentially relate to high expert hit rates.

4.3 Expert Prefetching

Given the searched and customized expert map $P_l^{(i)}$ for a layer $l \in [L]$ in Iteration i , we explain how it guides *FineMoE* to dynamically prefetch experts in fine granularity.

Similarity-aware expert selection. With the different contexts collected during iterations, expert maps searched by *FineMoE* also have varying similarity scores.⁵ Figures 8 and 9 demonstrated that similarity scores can effectively indicate the search confidence, where high searched similarity scores potentially mean high expert hit rates. Hence, we design *FineMoE*'s expert prefetching to be similarity-aware. For a layer $l \in [L]$ with a $\text{score} \in [-1, 1]$ to prefetch, we first dynamically compute an expert selection threshold $\delta_l \in [0, 1]$:

$$\delta_l := \text{Clip}(1 - \text{score}, 0, 1) = \max(0, \min(1 - \text{score}, 1)),$$

where score is the cosine similarity score computed in Equations 4 and 5. Given searched P_l , we find the set of experts to prefetch E_{prefetch} by iteratively picking the expert with the highest probability from $P_l = \{p_{l,1}, \dots, p_{l,j}, \dots, p_{l,J}\}$ until the

⁵In the following paper, we use “similarity scores” in both search contexts for simplicity, *i.e.*, semantic similarity in semantic-based expert map search and trajectory similarity in trajectory-based search, respectively.

summed probability of $E_{prefetch}$ exceeds δ_l :

$$\min_{\{E_{l,j}\}} |E_{prefetch}| \quad (6)$$

$$\text{s.t.} \quad \sum_{E_{l,j} \in E_{prefetch}} p_{l,j} \geq \delta_l, \quad j \in [J], \quad \forall l \in [L], \quad (7)$$

$$|E_{prefetch}| \geq K, \quad K \leq [J], \quad (8)$$

where K is the number of experts needed to activate per layer (e.g., Mixtral-8×7B activates two experts per layer). Constraint 7 requires the total probability of selected experts to prefetch per layer to be greater than δ_l . Constraint 8 represents the minimum number of selected experts must be larger than the number of experts to activate required by the MoE model. Intuitively, we assign a higher δ to low-score expert maps so that more experts are prefetched to mitigate mispredictions and assign a lower δ for high-score expert maps to reduce the memory footprint. Experts with higher probabilities are prioritized to be prefetched.

Asynchronous expert map searching and prefetching. Existing studies [16, 58] predict and prefetch experts synchronously during inference, severely hindering the inference performance. For example, MoE-Infinity [58] cannot compute forward functions before finishing expert prediction and prefetching at every MoE layer [59]. To minimize the system overhead and inference latency, we decouple the map searching and expert prefetching from the inference process using an asynchronous Publisher-Subscriber architecture (Figure 7). The Expert Map Store is a message broker that keeps messages from both the inference process and the Expert Map Searcher. As the inference proceeds, *FineMoE*'s inference process continuously publishes and writes the inference contexts (i.e., semantic embeddings and expert probability distributions) to the Expert Map Store. At the same time, the Expert Map Searcher subscribes to the context data, searches expert maps based on new context data, and prefetches experts to the Expert Cache in an asynchronous manner.

4.4 Expert Map Store Management

Practically, we design *FineMoE*'s Expert Map Store to maintain a capacity C for storing unique expert maps. To effectively guide inference across diverse prompts, it makes sense to identify and deduplicate redundant expert maps.

Expert map deduplication. Since *FineMoE* uses two approaches (i.e., semantic-based and trajectory-based) to compute similarity, we unify the two similarity scores to compute the pairwise redundancy scores between new iteration data and historical iteration data:

$$RDY_{x,y} := \frac{d}{L} \cdot score_{x,y}^{sem} + \frac{L-d}{L} \cdot score_{x,y}^{traj}, \quad x \in [B], \quad y \in [C],$$

where $score_{x,y}^{sem} \in \mathbb{R}^{B \times C}$ and $score_{x,y}^{traj} \in \mathbb{R}^{B \times C}$ are semantic-based and trajectory-based pairwise similarity scores calculated from Equations 4 and 5, d is the prefetch distance, L is

the total number of layers, B is the batch size of new interaction data, and C is the Expert Map Store capacity. Intuitively, as shown in Figure 7, the semantic-based and trajectory-based similarity scores contribute to the search expert map in proportion to $\frac{d}{L}$ and $\frac{L-d}{L}$, respectively. Therefore, we follow the same ratio to unify and compute the redundancy score. Whenever new iterations' context data arrive at the Expert Map Store, we compute the pairwise redundancy score $RDY_{x,y}$ to determine which old iterations to drop. Hence, we update the old iterations y (columns in $RDY_{x,y}$) with new iterations x (corresponding rows in $RDY_{x,y}$) in the Expert Map Store.

Theoretical analysis. The expert map deduplication can be formulated as a Minimum Sphere Covering problem [17]. Each expert map is a vectorized patch, and the full sphere represents all possible expert selections. The objective is to cover as much of the sphere as possible using a small number of maps, keeping storage overhead low. Studies [15, 46] have proved that maintaining at least $2LJ$ expert maps guarantees a lower bound of 75% expert map similarity (i.e., we can find an expert map that is at least 75% similar to any new iterations), and keeping $\frac{1}{2}LJ \ln(LJ)$ expert maps provides a lower bound of 98% similarity, where L and J are the numbers of layers and experts per layer in the MoE model, respectively. Given that modern MoE-based LLMs generally have $L \in [8, 128]$ and $J \in [24, 96]$, we can approximate the Expert Map Store's maximal requirement to be less than 50K expert maps with 200 MB CPU memory [58].

4.5 Expert Caching and Eviction

Similar to existing expert offloading solutions [16, 51, 58], we design *FineMoE* to maintain an Expert Cache on GPUs to reuse expert weights when serving different request prompts. Given searched expert maps from §4.2, we guide *FineMoE*'s Expert Cache to compute two priority scores for individual experts: 1) a *prefetching priority* to decide the orders to prefetch experts in the searched maps, and 2) an *eviction priority* to determine the orders to evict experts in the Expert Cache.

Expert prefetching priority. Recall the set of experts to prefetch $E_{prefetch}$ is determined in Equation 6. For each expert $E_{l,j} \in E_{prefetch}$, we define the prefetching priority to be

$$PRI_{l,j}^{prefetch} := \frac{p_{l,j}}{l - l_{now}}, \quad l \in [L], \quad j \in [J],$$

where $p_{l,j}$ is the expert probability from the searched expert map, and l_{now} is the current layer that the inference process stays at. Intuitively, experts with a higher probability $p_{l,j}$ to be activated should be prefetched sooner, and experts that sit closer to the current layer (i.e., smaller $l - l_{now}$) should also be prioritized.

Expert eviction priority. Similar to MoE-Infinity [58], *FineMoE*'s expert caching is based on the least frequently used (LFU) caching algorithm. We integrate the searched map to jointly determine the eviction priority. For each expert

$E_{l,j} \in E_{cache}$, we define the eviction priority to be

$$PRI_{l,j}^{evict} := \frac{1}{p_{l,j} \cdot freq_{l,j}}, \quad l \in [L], j \in [J],$$

where $freq_{l,j}$ is the cache visit frequency and $p_{l,j}$ is the probability from the searched map for an expert $E_{l,j} \in E_{cache}$. Intuitively, when reaching the Expert Cache limit, we want to first evict experts who are less frequently hit and have lower probabilities of being activated. Note that similar to existing works [51, 58], we do not consider the recent usage of experts as opposed to the classic least recently used (LRU) algorithm [16]. Since the expert usage is layer-wise sequential, *i.e.*, one layer following another, prioritizing recently used experts is against the nature of sequential forward computation.

On-demand expert loading. Mispredictions of expert prefetching lead to expert miss in the Expert Cache, as the MoE model cannot find available experts designated by the gate networks. Whenever an expert miss occurs, *FineMoE* pauses all expert prefetching tasks and immediately loads missed experts from CPU to GPU memory for fast serving.

5 *FineMoE*'s Implementation

We prototype *FineMoE* on top of Huggingface Transformers framework [55] using MoE-Infinity codebase [59]. The implementation of *FineMoE* is described as follows.

Expert Map Store is implemented in Python using PyTorch [42] and NumPy [19] libraries. We store all semantic embeddings and expert maps using `ndarrays` data structure for efficient array operations. The arrays are converted to tensors to compute similarity for expert map searching.

Expert Map Searcher is implemented in Python using PyTorch [42]. We implement the pairwise computations, including similarity (§4.2) and redundancy (§ 4.4) scores, using PyTorch native operations.

Expert Cache is implemented in C++ based on MoE-Infinity codebase [59]. The expert management in GPUs is implemented with the CUDA Runtime APIs [40]. We implement prefetching and caching logic of *FineMoE* in the MoE-Infinity codebase to enable expert offloading. Same with MoE-Infinity, *FineMoE* supports multi-GPU inference with expert parallelism (EP), where the experts are mapped to different GPU devices for loading and offloading. We use a hash map to assign expert IDs to different GPUs and retrieve them during inference. The expert assignment follows a round-robin manner to balance the overall GPU load. Additionally, we use a task pool in the GPU space with asynchronous threads to schedule and execute expert prefetching and on-demand loading tasks.

6 Evaluation

6.1 Experimental Setup

We introduce our evaluation methodology in this section.

Testbed. We conduct all experiments on a six-GPU testbed, where each GPU is an NVIDIA GeForce RTX 3090 with 24 GB GPU memory. All GPUs are interconnected using pairwise NVLinks and connected to the CPU memory using PCIe 4.0 with 32GB/s bandwidth. Additionally, the testbed has an AMD Ryzen Threadripper PRO 3955WX CPU with 32 cores and 480 GB CPU memory.

Models. We employ three popular MoE-based LLMs in our evaluation: Mixtral-8×7B [23], Qwen1.5-MoE [60], and Phi-3.5-MoE [1]. Table 1 describes the parameters, number of MoE layers, and number of experts per layer for the three models. Following the evaluation of existing works [51], we profile the models to set the optimal prefetch distance d to three before evaluation.

Datasets and traces. We employ two real-world prompt datasets commonly used for LLM evaluation: LMSYS-Chat-1M [64] and ShareGPT [49]. For most experiments, we split the sampled datasets in a standard 7:3 ratio, where 70% of the prompts' context data (*i.e.*, semantic embeddings and expert maps) are stored in *FineMoE*'s Expert Map Store, and 30% of the prompts are used for testing. For online serving experiments, we empty the Expert Map Store and use real-world LLM inference traces [43, 52] released by Microsoft Azure to set input and generation lengths and drive invocations.

Baselines. We compare *FineMoE* against four SOTA MoE serving baselines: 1) **MoE-Infinity** [58] uses coarse-grained request-level expert activation patterns and synchronous expert prediction and prefetching for MoE serving. We prepare the expert activation matrix collection for MoE-Infinity before evaluation for a fair comparison. 2) **ProMoE** [51] employs a stride-based speculative expert prefetching approach for MoE serving. Since the codebase of ProMoE is not open-sourced and requires training predictors for each MoE model, we reproduced a prototype of ProMoE on top of MoE-Infinity in our best effort. 3) **Mixtral-Offloading** [16] combines a layer-wise speculative expert prefetching and a LRU-based expert cache. 4) **DeepSpeed-Inference** [4] employs an expert-agnostic layer-wise parameter offloading approach, which uses pure on-demand loading and does not support prefetching. We implement the offloading logic of DeepSpeed-Inference in the MoE-Infinity codebase and add an expert cache for a fair comparison. We enable all baselines to serve MoE models from HuggingFace Transformer [55].

Metrics. Following the standard evaluation methodology of existing works [3, 51, 58, 65] on LLM serving, we report the performance of the prefill and decode stages separately. We measure Time-to-First-Token (TTFT) for the prefill stage and Time-Per-Output-Token (TPOT) for the decode stage. Additionally, we also report other system metrics, such as expert hit rate and overheads, for detailed evaluation.

6.2 Offline Serving Performance

We first evaluate the offline serving performance of prefill and decode stages when running *FineMoE* and other baselines

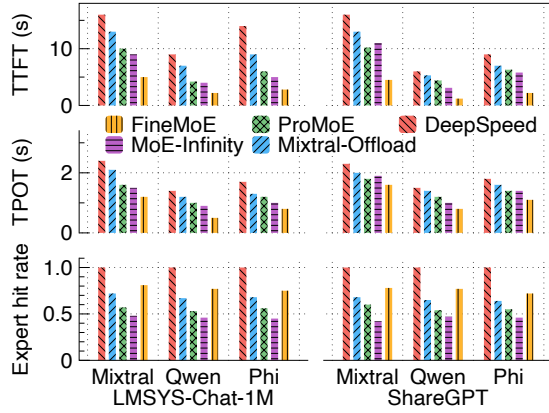


Figure 10. Overall performance of prefill and decode stages.

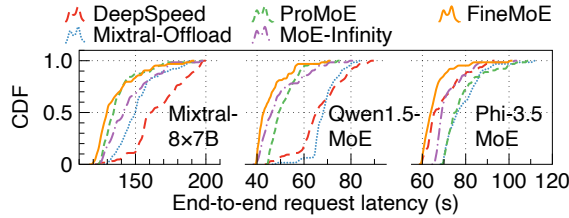


Figure 11. CDF of request latency for MoE online serving.

with the three MoE models, where we report Time-To-First-Token (TTFT) and Time-Per-Output-Token (TPOT). Similar to existing works [3, 65], we measure TTFT and TPOT for individual prompts for each combination of model and dataset. For evaluation with LMSYS-Chat-1M and ShareGPT datasets, the input lengths are set to 37 and 43 tokens, and generation lengths to 127 and 122 tokens, which are the mean values calculated across datasets, respectively. For each dataset, we randomly sample 64 prompts and report average results.

Figure 10 shows the TTFT, TPOT, and expert hit rate of *FineMoE* and four baselines when serving three MoE models with LMSYS-Chat-1M and ShareGPT datasets, respectively. DeepSpeed-Inference has both the worst TTFT and TPOT due to expert-agnostic offloading and lacking expert prefetching. While Mixtral-Offloading, ProMoE, and MoE-Infinity perform better than DeepSpeed-Inference, they are underperformed by *FineMoE* because of coarse-grained offloading designs. Compared to DeepSpeed-Inference, Mixtral-Offloading, ProMoE, and MoE-Infinity, *FineMoE* reduces the average TTFT by 74%, 67%, 56%, and 53%, and reduces the average TPOT by 46%, 38%, 27%, and 22%, respectively.

For expert hit rate, DeepSpeed-Inference has no expert misses because it fetches whole layers with full experts, but with the worst latency due to pure on-demand loading. Mixtral-Offloading achieves a higher hit rate than ProMoE and MoE-Infinity because of its synchronous speculative

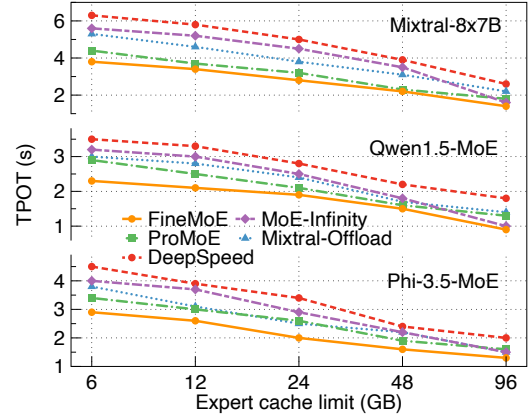


Figure 12. Performance under varying expert cache limits.

prefetching with a prefetch distance of 1. However, due to synchronous prefetching, its TTFT and TPOT are worse than others except DeepSpeed-Inference. Overall, *FineMoE* improves the average expert hit rate by 14%, 37%, and 68% over Mixtral-Offloading, ProMoE, and MoE-Infinity, respectively.

6.3 Online Serving Performance

Except for the offline evaluation (*i.e.*, Expert Map Store in full capacity before serving), we also evaluate *FineMoE* against other baselines in online serving settings. We empty the Expert Map Store of *FineMoE* and the expert activation matrix collection of MoE-Infinity for the online serving experiment. The request traces are derived from Azure LLM inference traces [43, 52], with randomly sampled 256 requests (2.91 requests per second), to drive LMSYS-Chat-1M prompts for each MoE model serving. To ensure consistency, *FineMoE* and all baselines input and generate the exact number of tokens specified in the traces. Figure 11 illustrates the CDF of end-to-end request latency across three MoE models. The results demonstrate that *FineMoE* significantly reduces overall request latency compared to other baselines in online serving.

6.4 Impact of Expert Cache Limits

We measure the TPOT of *FineMoE* and other baselines by limiting the expert cache memory budget to investigate their performance in the latency-memory trade-off (§2.3). We mainly focus on TPOT to show the end-to-end performance impacted by varying cache limits. Figure 12 shows the TPOT of *FineMoE* and four baselines when serving three MoE models under different expert cache limits. We gradually increase the GPU memory allocated for caching experts from 6 GB to 96 GB while employing the same experimental setting in §6.2. Similarly, DeepSpeed-Inference has the worst TPOT due to being expert-agnostic. *FineMoE* consistently outperforms Mixtral-Offloading, ProMoE, and MoE-Infinity under varying expert cache limits. As the cache limit increases, the performance gap between all baselines narrows due to the

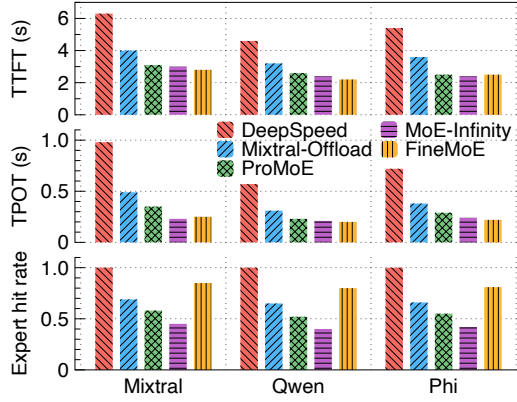
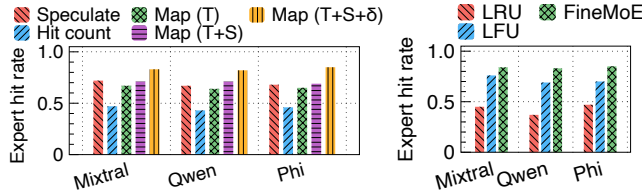


Figure 13. Performance on high-end GPU testbed.



(a) Expert pattern tracking approaches. (b) Prefetch and caching.

Figure 14. Ablation study of *FineMoE*.

increased availability of cached experts. Nevertheless, for limited GPU memory sizes (e.g., 6GB), *FineMoE* reduces the TPOT by 36%, 25%, 16%, and 29%, compared to DeepSpeed-Inference, Mixtral-Offloading, ProMoE, and MoE-Infinity, across three MoE models, respectively. With fine-grained expert offloading, *FineMoE* significantly reduces the expert on-demand loading latency while maintaining a lower GPU memory footprint, therefore achieving a better spot in the latency-memory trade-off of MoE serving.

6.5 Impact of GPU Performance

To evaluate the impact of GPU performance on offloading methods, we repeat the experiments using LMSYS-Chat-1M on an NVIDIA A100 testbed equipped with 80 GB of HBM2e memory and a peak memory bandwidth of 2 TB/s. Figure 13 presents the serving performance of *FineMoE* and the baselines across the three MoE models. *FineMoE* achieves smaller performance gains on the A100 than on the 6×3090 testbed, since high-end GPUs and the lack of EP yield faster inference and lower offloading overhead. Nevertheless, *FineMoE* consistently outperforms all baselines. The expert hit rate remains largely unaffected, as GPU performance has less impact on expert predictions.

6.6 Ablation Study

We present the ablation study of *FineMoE*'s design.

Effectiveness of expert map search. One of *FineMoE*'s key designs is the expert map, which tracks expert selection

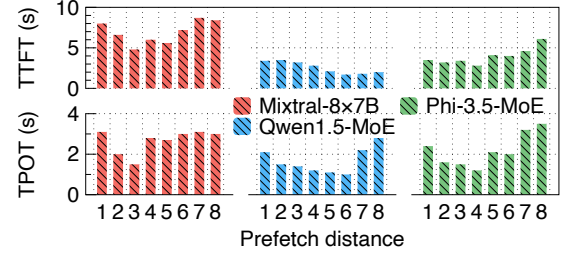
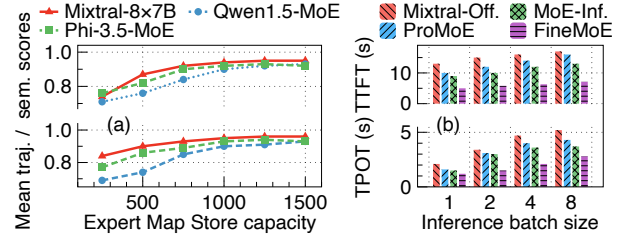


Figure 15. Performance with different prefetch distances.

Figure 16. Sensitivity analysis of *FineMoE*.

preferences in fine granularity. We evaluate the effectiveness of the expert map against five expert pattern-tracking approaches as follows. 1) **Speculate**: speculative prediction used by Mixtral-Offloading [16] and ProMoE [51], 2) **Hit count**: request-level expert hit count used by MoE-Infinity [58], 3) **Map (T)**: expert map with only trajectory similarity search, 4) **Map (T+S)**: expert map with both trajectory and semantic similarity search but statically select top-K experts to prefetch, and 5) **Map (T+S+δ)**: expert map with full features enabled, including trajectory and semantic similarity search (§4.2) and dynamically selecting experts to prefetch (§4.3). We implement the above methods in *FineMoE*'s Expert Map Searcher for a fair comparison. Figure 14a shows the expert hit rate of the above expert pattern tracking methods. Speculative prediction is effective due to the widespread presence of residual connections in Transformer blocks. However, its effectiveness decreases drastically as prefetch distance increases [51]. The request-level expert activation count has the worst performance due to coarse granularity. As features are incrementally restored to *FineMoE*'s expert map, the expert hit rate gradually increases, demonstrating its effectiveness.

Effectiveness of expert prefetching and caching. We evaluate *FineMoE*'s expert prefetching and caching against two caching algorithms: 1) **LRU** used by Mixtral-Offloading [16] and 2) **LFU** used by MoE-Infinity [58]. Figure 14b depicts the expert hit rate of *FineMoE* and two baselines. The results show that LRU performs poorly in expert offloading scenarios. Though LFU achieves a higher hit rate than LRU, *FineMoE* surpasses both, achieving the highest expert hit rate.

6.7 Sensitivity Analysis

We analyze the sensitivity of prefetch distance of MoE models, Expert Map Store capacity, and inference batch size.

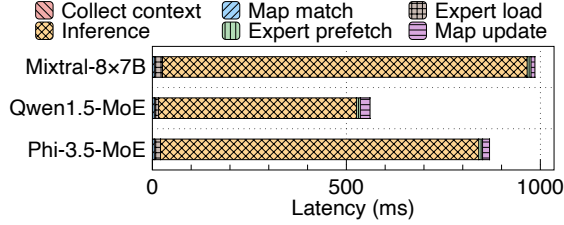


Figure 17. Latency breakdown of *FineMoE*'s one iteration.

Prefetch distance of MoE models. Figure 15 shows the TTFT and TPOT of *FineMoE* when serving three MoE models with different prefetch distances. We have demonstrated that the expert hit rate decreases when gradually increasing the prefetch distance (Figure 4). When the prefetch distance is small, *FineMoE* cannot perfectly hide its system delay from the inference process, such as the map searching and expert prefetching, leading to an increase in inference latency. With larger prefetch distances, *FineMoE* has worse expert hit rates that also degrade performance. Therefore, we set the prefetch distance d to 3, 6, and 4 for Mixtral-8x7B, Qwen1.5-MoE, and Phi-3.5-MoE, respectively.

Capacity of Expert Map Store. We measure the mean semantic and trajectory similarity scores searched in *FineMoE*'s expert map searching for MoE model serving. Figure 16(a) presents the mean semantic and trajectory similarity scores of *FineMoE* with different Expert Map Store capacity sizes. Both semantic and trajectory similarity scores improve as the store capacity increases. While the similarity scores exhibit a significant increase with capacities below 1K, further capacity expansion yields diminishing similarity gains. To minimize *FineMoE*'s memory overhead, we set *FineMoE*'s Expert Map Store capacity to 1K in evaluation.

Inference batch size. We investigate the impact of inference batch size on *FineMoE* and three baselines using Mixtral-8x7B with LMSYS-Chat-1M. Figure 16(b) presents the performance of *FineMoE*, Mixtral-Offloading, ProMoE, and MoE-Infinity as the batch size increases from one to eight. *FineMoE* achieves the lowest TTFT and TPOT in most cases.

6.8 System Overheads

We measure and report the system overheads of *FineMoE*.

Latency overheads of *FineMoE*'s operations. Figure 17 shows the latency breakdown of one inference iteration in *FineMoE* when serving the three MoE models. We report operation overheads of *FineMoE*, including context collection, map searching, expert on-demand loading, expert prefetching, and map update after the iteration completes. Qwen1.5-MoE has lower end-to-end iteration latency than Mixtral-8x7B and Phi-3.5-MoE because of significantly fewer parameters. Note that expert prefetching, map searching, and map update tasks are executed asynchronously, aside from the inference process. Hence, they do not contribute to the end-to-end iteration latency. Excluding three asynchronous tasks, the total

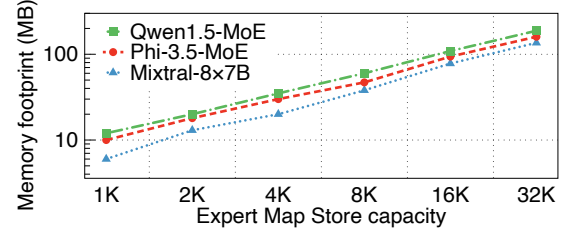


Figure 18. CPU memory footprint of *FineMoE*'s Expert Map Store with different capacity.

delay incurred by other operations is consistently less than 50ms (1% of the iteration) across three MoE models, which is negligible compared to the inference latency.

Memory overheads of *FineMoE*'s Expert Map Store. Figure 18 shows the CPU memory footprint of *FineMoE*'s Expert Map Store when varying the store capacity from 1K to 32K maps. The memory needed to store expert maps for Qwen1.5-MoE is more than Mixtral-8x7B and Phi-3.5-MoE because it has more experts per layer over the other two models, which increases the map shape. Even for the largest capacity (32K), the Expert Map Store requires less than 200MB of memory to store the maps, which is trivial since modern GPU servers usually have abundant CPU memory (e.g., p4d.24xlarge on AWS EC2 [5] has over 1100 GB of CPU memory). In evaluation, *FineMoE*'s map store capacity with 1K maps is sufficient for maintaining performance (§6.7), resulting in minimal memory overhead.

7 Discussion

In this section, we compare the heuristic-based *FineMoE* with Neural Network (NN)-based predictors, analyze the impact of model parallelism on *FineMoE*'s performance, and discuss how *FineMoE* can be extended to other MoE architectures.

NN-based predictors. NN-based predictors for expert offloading are impractical due to multiple sources of overhead. First, they often introduce sub-second inference latency, comparable to MoE inference latency itself. Second, they require extensive data collection, hour-long per-layer training, and frequent retraining to adapt to workload shifts. Third, they consume substantial GPU memory, as prior work [51] reports millions of parameters per MoE layer. Moreover, they are incompatible with *FineMoE*'s fine-grained design: training on fine-grained data hinders convergence, while storing iteration-level probabilities generates large volumes that further prolong training and limit feasibility. Therefore, we adopt a heuristic-based design rather than NN-based approaches.

Impact of EP and tensor parallelism (TP). Higher EP distributes experts across more devices and enables greater expert replication, which can increase *FineMoE*'s offloading opportunities and memory savings. In contrast, higher TP raises the overhead of offloading operations, since dense model components are split across devices and require coordinated offloading and reloading. As noted in prior work [11, 35],

production MoE systems generally avoid high TP because its communication costs outweigh performance benefits. In large-scale deployments (e.g., DeepSeek [35]), MoE systems usually use low EP during prefill to maximize throughput, while adopting high EP during decode to enable higher expert redundancy. Though high-EP decode reduces per-GPU expert occupancy, the larger number of expert replicas (e.g., 2× more than prefill in DeepSeek [35]) creates additional offloading opportunities by allowing experts to be compacted onto fewer devices.

Adaptation to other MoE architectures. *FineMoE* can be easily integrated with different MoE architectures. For shared experts, we treat them as always-hit during expert prediction. One of our evaluated models, Qwen1.5-MoE, includes shared experts that are used by all tokens. For multi-gating MoE, we can extend the expert map search by recording each gate’s probability distribution and flattening the outputs into a single vector for efficient similarity computation. This enables unified handling across diverse routing schemes.

8 Related Work

In this section, we provide a brief overview of recent studies and related works on MoE serving.

Lossless MoE serving. Recent studies on lossless MoE serving have been widely proposed. DeepSpeed Inference [4] offloads layer-wise parameters without considering expert awareness and does not provide expert prefetching or caching capabilities. Mixtral-Offloading [16] employs LRU expert caching and introduces speculative prediction to enable expert prefetching. MoE-Infinity [58] proposes the request-level expert activation matrix to guide offloading in coarse granularity. SwapMoE [47] maintains a set of critical experts in GPU memory and adjusts them based on workload changes to minimize offloading overhead. ProMoE [51] trains predictors per MoE layer to achieve high speculative prediction accuracy and low inference latency. Lina [32] exports unpopular experts to host memory while focusing on MoE training. Liu et al. [36] partitions and serves MoE models on serverless computing. Fiddler [26] serves MoE inference on CPU and GPU collaboratively. MoEShard [6] shards experts to achieve balanced expert loads. Unlike existing coarse-grained offloading solutions, *FineMoE* tracks fine-grained expert patterns from both trajectory and semantic aspects and outperforms SOTA baselines.

Lossy MoE serving. Expert pruning [13] reduces memory usage by removing under-utilized experts. Expert compression [21, 44, 53, 56, 66] compresses less-popular experts to reduce models’ memory footprint. Expert load rerouting [18, 20, 62] balances tokens to under-loaded experts instead of following the outputs of gate networks. Specifically, Hobbit [53] uses low precision to serve less-critical experts.

He et al. [20] drops and reroutes tokens from overloaded experts to others to reduce the straggler effect. Samoyeds [56] serves MoE models with sparsity computing. Lynx [18] selects experts based on batch-level expert importance instead of gate network outputs. DAOP [62] performs computations with predicted experts directly and cannot guarantee full generation quality. FLoE [66] compresses experts on-the-fly during inference. However, lossy serving impacts the model quality and is orthogonal to *FineMoE*.

MoE refactorization. Some works propose to redesign and refactor the current MoE architecture, such as decoupling gate networks from inference process [14, 22] or building activation-efficient MoE models [8, 25]. This line of work requires model training or fine-tuning before serving. Pre-gated MoE [22] trains pre-gate functions to eliminate the sequential dependencies between expert selection and execution. SiDA [14] proposes a sparsity-inspired data-aware inference system that decouples the expert routing from inference. READ-ME [8] refactors pre-trained dense LLMs into specialized MoE models. MoLE [25] replaces inputs of all MoE layers with embedding tokens to avoid sparse expert activation. In contrast, *FineMoE* requires zero training to serve MoE models.

9 Conclusion

This paper proposes *FineMoE*, a fine-grained expert offloading system for MoE serving that achieves low inference latency without incurring significant model memory footprints. *FineMoE* tracks iteration-level expert probability distributions from the MoE model using expert map and analyzes input semantic embeddings from individual request prompts. Based on the input semantic and expert trajectory information, *FineMoE* searches the most accurate expert map to carefully guide the expert prefetching, caching, and offloading decisions tailored to every inference iteration. *FineMoE* is prototyped on top of HuggingFace Transformers and deployed to a six-GPU testbed. Extensive experiments with open-source MoE models and real-world workloads show that *FineMoE* reduces inference latency by 47% and improves expert hit rate by 39% compared to state-of-the-art solutions.

Acknowledgments

We thank anonymous reviewers and our shepherd, Dr. Yaniv David, for their valuable feedback. The work of Hanfei Yu and Hao Wang was supported in part by NSF 2527416, 2534241, and 2523997, and the AWS Cloud Credit for Research program. The work of Hao Wang (Rutgers CS) was supported in part by Amazon Faculty Research Award, Microsoft AI & Society Fellowship, NSF CAREER Award IIS-2340125, NIH grant R01CA297832, and NSF grant IIS-2127918. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the funding agencies.

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. *arXiv preprint arXiv:2404.14219* (2024).
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Ameet Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*.
- [5] AWS. 2006. AWS EC2: Secure and Resizable Compute Capacity in the Cloud. <https://aws.amazon.com/ec2/>.
- [6] Oana Balmau, Anne-Marie Kermarrec, Rafael Pires, André Loureiro Espírito Santo, Martijn de Vos, and Milos Vujanovic. 2025. Accelerating MoE Model Inference with Expert Sharding. In *Proceedings of the 5th Workshop on Machine Learning and Systems (EuroMLSys)*.
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. *Advances in neural information processing systems* (2020).
- [8] Ruisi Cai, Yeonju Ro, Geon-Woo Kim, Peihao Wang, Babak Ehteshami Bejnordi, Aditya Akella, and Zhangyang Wang. 2024. Read-ME: Refactorizing LLMs as Router-Decoupled Mixture of Experts with System Co-Design. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS)*.
- [9] Israel Cohen, Yiteng Huang, Jingdong Chen, Jacob Benesty, Jacob Benesty, Jingdong Chen, Yiteng Huang, and Israel Cohen. 2009. Pearson Correlation Coefficient. *Noise Reduction in Speech Processing* (2009).
- [10] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. 2022. *Introduction to Algorithms*. MIT press.
- [11] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y Wu, et al. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *arXiv preprint arXiv:2401.06066* (2024).
- [12] Sunhao Dai, Yuqi Zhou, Liang Pang, Weihao Liu, Xiaolin Hu, Yong Liu, Xiao Zhang, Gang Wang, and Jun Xu. 2024. Neural Retrievers are Biased Towards LLM-Generated Content. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.
- [13] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations (ICLR)*.
- [14] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Li, and Yiran Chen. 2024. SiDA: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models. *Proceedings of Machine Learning and Systems (MLSys)* (2024).
- [15] Ilya Dumer. 2007. Covering Spheres with Spheres. *Discrete & Computational Geometry* (2007).
- [16] Artyom Eliseev and Denis Mazur. 2023. Fast Inference of Mixture-of-Experts Language Models with Offloading. *arXiv preprint arXiv:2312.17238* (2023).
- [17] D Jack Elzinga and Donald W Hearn. 1972. The Minimum Covering Sphere Problem. *Management Science* (1972).
- [18] Vima Gupta, Kartik Sinha, Ada Gavrilovska, and Anand Padmanabha Iyer. 2024. Lynx: Enabling Efficient MoE Inference through Dynamic Batch-Aware Expert Selection. *arXiv preprint arXiv:2411.08982* (2024).
- [19] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array Programming with NumPy. *Nature* (2020).
- [20] Shwai He, Weilin Cai, Jiayi Huang, and Ang Li. 2025. Capacity-Aware Inference: Mitigating the Straggler Effect in Mixture of Experts. *arXiv preprint arXiv:2503.05066* (2025).
- [21] Shwai He, Daize Dong, Liang Ding, and Ang Li. 2024. Towards Efficient Mixture of Experts: A Holistic Study of Compression Techniques. *arXiv preprint arXiv:2406.02500* (2024).
- [22] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*.
- [23] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024).
- [24] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R Lyu. 2024. LILAC: Log Parsing using LLMs with Adaptive Parsing Cache. *Proceedings of the ACM on Software Engineering* (2024).
- [25] Shibo Jie, Yehui Tang, Kai Han, Yitong Li, Duyu Tang, Zhi-Hong Deng, and Yunhe Wang. 2025. Mixture of Lookup Experts. In *International Conference on Machine Learning (ICML)*.
- [26] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2025. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. In *International Conference on Learning Representations (ICLR)*.
- [27] Young Jin Kim, Raffi Fahim, and Hany Hassan Awadalla. 2023. Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness. *arXiv preprint arXiv:2310.02410* (2023).
- [28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.
- [29] Jaekyu Lee, Hyesoon Kim, and Richard Vuduc. 2012. When Prefetching Works, When It Doesn't, and Why. *ACM Transactions on Architecture and Code Optimization (TACO)* (2012).
- [30] Jaeseong Lee, Aurick Qiao, Daniel F Campos, Zhewei Yao, Yuxiong He, et al. 2024. STUN: Structured-Then-Unstructured Pruning for Scalable MoE Pruning. *arXiv preprint arXiv:2409.06211* (2024).
- [31] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [32] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating Distributed MoE training and inference with Lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*.
- [33] Yichen Li, Yintong Huo, Renyi Zhong, Zhihan Jiang, Jinyang Liu, Junjie Huang, Jiazhen Gu, Pinjia He, and Michael R Lyu. 2024. Go Static: Contextualized Logging Statement Generation. *Proceedings of*

- the ACM on Software Engineering* (2024).
- [34] Xinyu Lin, Wenjie Wang, Yongqi Li, Shuo Yang, Fuli Feng, Yinwei Wei, and Tat-Seng Chua. 2024. Data-efficient Fine-tuning for LLM-based Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
 - [35] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).
 - [36] Mengfan Liu, Wei Wang, and Chuan Wu. 2025. Optimizing Distributed Deployment of Mixture-of-Experts Model Inference in Serverless Computing. In *IEEE Conference on Computer Communications (INFOCOM)*.
 - [37] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*.
 - [38] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. (2013).
 - [39] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to Help with Code Understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*.
 - [40] NVIDIA. 2024. CUDA Runtime API :: CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/cuda-runtime-api/index.html>.
 - [41] Ollama. 2024. Get Up and Running with Large Language Models. <https://ollama.com/>.
 - [42] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems (NIPS)* (2019).
 - [43] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*.
 - [44] Pingzhi Li, Zhenyu Zhang, Prateek Yadav, Yi-Lin Sung, Yu Cheng, Mohit Bansal, Tianlong Chen. 2024. Merge, Then Compress: Demystify Efficient SMOE with Hints from Its Routing Policy. In *International Conference on Learning Representations (ICLR)*.
 - [45] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language Models are Unsupervised Multi-task Learners. *OpenAI blog* (2019).
 - [46] Robert Alexander Rankin. 1947. On the Closest Packing of Spheres in N Dimensions. *Annals of Mathematics* (1947).
 - [47] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, Yunxin Liu. 2023. SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget. *arXiv preprint arXiv:2308.15030* (2023).
 - [48] Claude Elwood Shannon. 1948. A Mathematical Theory of Communication. *The Bell System Technical Journal* (1948).
 - [49] ShareGPT. 2022. ShareGPT: Share Your Wildest ChatGPT Conversations. <https://sharegpt.com/>.
 - [50] Snowflake. 2024. Snowflake Arctic: The Best LLM for Enterprise AI. <https://www.snowflake.com/en/data-cloud/arctic/>.
 - [51] Xiaoniu Song, Zihang Zhong, and Rong Chen. 2024. ProMoE: Fast MoE-based LLM Serving using Proactive Caching. *arXiv preprint arXiv:2410.22134* (2024).
 - [52] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *International Symposium on High-Performance Computer Architecture (HPCA)*.
 - [53] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. Hobbit: A Mixed Precision Expert Offloading System for Fast MoE Inference. *arXiv preprint arXiv:2411.01433* (2024).
 - [54] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
 - [55] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Hugging-Face's Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*.
 - [56] Chenpeng Wu, Qiqi Gu, Heng Shi, Jianguo Yao, and Haibing Guan. 2025. Samoyeds: Accelerating MoE Models with Structured Sparsity Leveraging Sparse Tensor Cores. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*.
 - [57] xAI. 2023. Announcing Grok. <https://x.ai/blog/grok>.
 - [58] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. *arXiv preprint arXiv:2401.14361* (2024).
 - [59] Xue, Leyang and Fu, Yao and Lu, Zhan and Mai, Luo and Marina, Mahesh. [n. d.]. MoE-Infinity Codebase. <https://github.com/TorchMoE/MoE-Infinity>.
 - [60] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).
 - [61] Seniha Esen Yuksel, Joseph N Wilson, and Paul D Gader. 2012. Twenty Years of Mixture of Experts. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)* (2012).
 - [62] Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. 2025. DAOP: Data-Aware Offloading and Predictive Pre-Calculation for Efficient MoE Inference. In *Design Automation and Test in Europe (DATE)*.
 - [63] Yuyue Zhao, Jiancan Wu, Xiang Wang, Wei Tang, Dingxian Wang, and Maarten de Rijke. 2024. Let Me Do It for You: Towards LLM Empowered Recommendation via Tool Learning. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
 - [64] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P Xing, et al. 2023. LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset. *arXiv preprint arXiv:2309.11998* (2023).
 - [65] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
 - [66] Yuxin Zhou, Zheng Li, Jun Zhang, Jue Wang, Yiping Wang, Zhongle Xie, Ke Chen, and Lidan Shou. 2025. FloE: On-the-Fly MoE Inference on Memory-constrained GPU. In *International Conference on Machine Learning (ICML)*.